

Подготовка к экзамену по C++ (ИТМО)

Тема: CRTP и Роль Преобразования Типов

Hardcore Theory Edition

20 июня 2026 г.

1 Проблема динамического полиморфизма

Классический полиморфизм в C++ строится на виртуальных функциях (`virtual`). Как это работает: компилятор создает в памяти **vtable** (таблицу виртуальных функций). При вызове метода программа сначала идет в объект, берет оттуда скрытый указатель `vptr`, идет по нему в таблицу, находит там адрес нужной функции и только потом делает прыжок к коду.

- **Минусы:** Это медленно. Двойное разыменование указателя убивает кэш процессора (Cache Miss). А главное — компилятор **не может заинлайнить** (встроить код вместо вызова) виртуальную функцию, так как до этапа выполнения (Runtime) не знает, какой именно наследник будет вызван.

2 CRTP: Статический Полиморфизм

Curiously Recurring Template Pattern (Странно повторяющийся шаблон) позволяет делать полиморфизм на этапе компиляции, вообще без таблиц виртуальных функций!

Суть синтаксиса: Наследник наследуется от базового класса, передавая **самого себя** в качестве параметра шаблона.

```
1 template <typename Derived>
2 class Base { ... };
3
4 class MyClass : public Base<MyClass> { ... };
```

3 Магия `static_cast`

Как базовому классу вызвать метод наследника, если в базе нет виртуальных функций? Базовый класс **знает тип наследника** (он передан как шаблонный параметр `Derived`). Поэтому внутри базового класса мы берем указатель `this` (который имеет тип `Base*`) и **жестко кастуем его вниз** (Downcast) к типу наследника:

```
1 void interface_method() {
2     // Безопасный downcast на этапе компиляции!
3     Derived* child = static_cast<Derived*>(this);
4     child->implementation();
5 }
```

Почему мы используем `static_cast`, а не `dynamic_cast`? Потому что мы **гарантированно** знаем, что `this` является частью объекта `Derived` (благодаря наследованию). `static_cast` ничего не проверяет в рантайме, он работает со скоростью света. Компилятор видит точный тип и мгновенно инлайнит вызов. Zero Overhead!

Суть на пальцах (Жизненная аналогия)

Динамический полиморфизм (virtual): Вы звоните в справочную больницы (Base) и говорите "Позовите врача". Диспетчер лезет в журнал (vtable), ищет, кто сегодня дежурит, и переводит звонок на хирурга (Derived). Это занимает время.

Статический полиморфизм (CRTP): Вы приходите в кабинет, на котором висит табличка "Кабинет Хирурга Иванова" (Base<Ivanov>). Вы точно знаете, кто там сидит (`static_cast`). Никакой справочной не нужно, вы просто открываете дверь и говорите с Ивановым. Вызов мгновенный.

4 Где это используется?

- `std::enable_shared_from_this` — классический пример CRTP в стандартной библиотеке.
- **Библиотеки линейной алгебры (Eigen)** — математика требует огромной скорости, виртуальные функции там недопустимы.
- **Method Chaining (Fluent Interface)** — когда методы базового класса должны возвращать `Derived&`, чтобы можно было писать `obj.setX(1).setY(2);`.